Assignment I

Purpose

Design and implement a small functional application in the domain of your choice, in one of the following programming languages: Rust, C++, Python or Java. The components of the application should be integrated well in order to achieve some “useful” functionality while protecting resources owned by different users.

Your objective will be to build a small but secure application, challenge its trust assumptions and improve it. You will be investigating authorization across component boundaries and how composition of different services may lead to vulnerabilities that you did not anticipate during the design phase. Feel free to take inspiration for visualization and the report from the following example repository:
https://github.com/Abhi-Vaish/ISRM_Group-4/blob/main/README.md

Scope

- The application should have a multi-tier architecture: its components run separately and interact with each other through network or system interfaces, such as an HTTP API or a message queue (see Required architecture). Before you start implementing, identify the functional and non-functional requirements of the software and map out its data flow. For example, you have created a website which is connected to the database and has a URL for the user to connect to the application for the use of the application.
- The application must implement authentication and authorization enforcement, as well as at least three (3) functionalities, at least one of which involves all three components.
- There must be at least three (3) vulnerabilities intentionally planted from the course’s vulnerability catalog https://brightspace.au.dk/d2l/le/lessons/219647/topics/2943887, plus one (1) additional vulnerability stemming from composing the different components of your software. All four “planted” vulnerabilities must be documented in your report.

Choosing your application (Domains)

Choose the purpose, users, data model, rules, interfaces, and responsibilities carefully! Some examples include

Application	Protected resources	External dependencies
Bank	Loan requests	Math library
Booking	Bookings	Visualizer library

Research workspace	Private datasets and results	Export library

These are examples, not prescribed designs. Any other domain is equally acceptable if it meets the requirements below.

Use synthetic data, seeded accounts, and one or two main functional entity types. Account registration, a graphical interface, real payments, email delivery, public hosting, high availability, and large-scale performance **are outside of scope and do not need to be implemented**. A command-line client or API test collection is sufficient as the user interface.

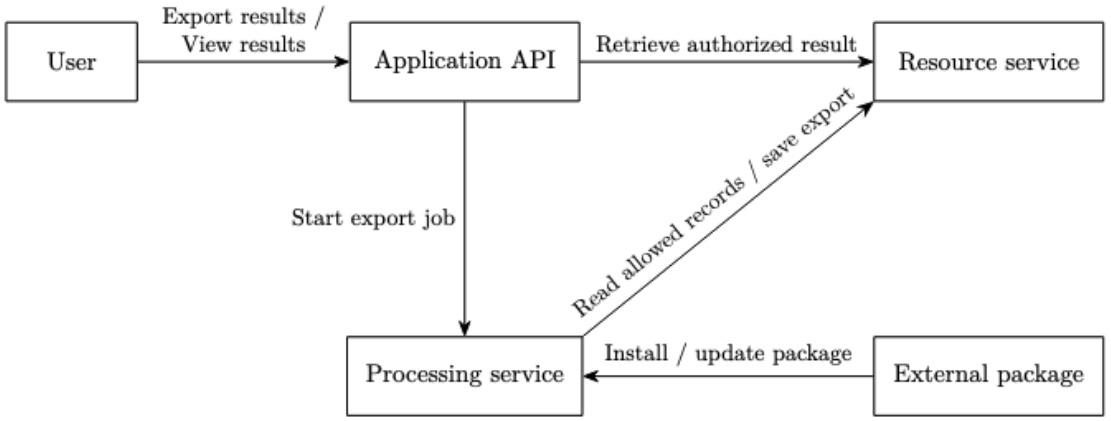
Required architecture

Implement three application components that run as separate processes (e.g., separate containers) and communicate through explicit interfaces such as HTTP or messages. Three classes within one process **do not meet this requirement**, and a database or a browser **does not count as one of the three components!** In addition, create or adapt one small helper package of your own (the “external” dependency) that one of your components uses. It can be as small as an integer-to-binary converter, but it must be declared and installed as a dependency of the project, not copied into a component’s source code.

At least one functionality must involve all three components. Each component should have different responsibilities and permissions. Consider one possible arrangement. One possible arrangement is shown below:

Component	Example responsibility	Security responsibility
Application API	Authenticate users and coordinate requests	Establish the user identity and authority of the requested operation
Resource service	Owns protected records and results	Enforce permissions for each resource and actions, including ones arriving from other services
Processing service	Perform some job using an external dependency	Operate only on the data requested for a specific job

For example a user could request an export using the application API. The application API starts an export job on the processing service, which reads only the records that user is allowed to access from the resource service, generates the export using the external package, and saves the result back to the resource service. The user then retrieves the export through the application API, and only the authorized user can do so (see Figure 1).



Authorization requirements

Use at least two user roles, including one ordinary role and one privileged role (e.g., admin) to perform some restricted action. Seed at least two ordinary users (so you can test that one user cannot access another user's resources) and one authorized.

You may use established session/token libraries. Explain how your mechanism binds authority to the relevant service, resource, action, and validity period. Custom cryptographic algorithms are not required nor advised (but in case you include any make sure to document them :) – your TA).

Apply at least three secure design principles from the course (e.g., least privilege, complete mediation, fail-safe defaults). For each, show a design decision, how it is implemented, and a test or other evidence.

Controlled supply chain attack (Optional)

Create or adapt a small helper package (the external dependency) that one of the components relies on during execution of one of its functionalities. Treat it as externally supplied software: that is it should be in a separate package directory or repository and be distributed as an installable artifact (e.g., a wheel, crate, JAR, or library file).

Prepare two versions of the package: a benign version and a deliberately malicious version (the "malicious fixture") used for testing. The malicious version should preserve the package's normal functionality, but at the same time try to access information it should not have access to, e.g., read another user's record or modify protected files outside of its jobs.

The malicious code must enter through the package installation, build or update path not by editing the consuming component's code directly.

Experiment A (Optional)

Store the approved package's name, source, version and content digest (e.g., a SHA-256 hash) in a manifest controlled by the consumer (your component), not by the supplier (of the "external" dependency). Make sure that the build or installation phase verifies the package against this manifest. You do not need a public package registry, a local directory or private repository is enough.

Replace the supplier artifact with different bytes while keeping the expected identity/version unchanged. Demonstrate that the integrity gate blocks its use. Keep the approval manifest outside the supplied software write permissions!

Experiment B (Optional)

In an isolated experimental run run (e.g., a separate branch or environment), explicitly approve the malicious fixtures identity and digest as a new authentic version. This models a compromised supplier releasing an update that the user accepts.

Now trigger the normal workflow and show that the malicious code executes and attempts its unauthorized action. Determine what the compromised component can access using its actual credentials, network reachability, files, and volumes.

The final implementation must prevent the selected action from accessing or modifying resources outside the documented authority. If it succeeded initially, preserve the evidence, correct the boundary, and repeat the experiment. Also confirm that the legitimate job still completes with the benign dependency.

Use an actual authorization, isolation or output-validation to mitigate the attack. As checking only the packages filename or version, is not a sufficient runtime defence.

SSDLC and CI/CD

Before feature implementation, commit a short plan covering all four phases to your repository. Reuse the approach from the previous threat-modelling and DevSecOps exercise:

Asset → threat → requirement/control → lifecycle activity → test/evidence → gate decision.

Phase	Required deliverable	Gate
Requirements	Scope and operation list, asset register, component/trust-boundaries , access control matrix, STRIDE analysis	Review that the architecture meets the scope, authorities are defined, and relevant threats have controls.
Implementation and build	Three working components, reproducible builds, dependencies	Unexpected or unapproved artifacts (e.g., unlisted dependencies) artefacts are rejected.
Testing	Functional and authorization tests, investigation and fixes	Required functional and security tests pass. Malformed artefacts artefacts are rejected.
Release, deploy and operation	Discuss in the report what a production release would need.	-

Implement a Jenkins pipeline that runs on every push and pull request and:

1. Builds or validates the components, run static analysis and dependency analysis
2. Starts a fresh test environment and runs functional and authorization tests.
3. Saves test and analysis results as artifacys.

Define which findings block a release and configure the pipeline to fail on them. Correct confirmed violations of the required security guarantees.

Written report

Write around 2,000-2,500 words (around 7 pages total), excluding diagrams, tables, references, code excerpts, and appendices. We encourage everyone to include diagrams, tables and other kinds of data representation! Organize your report around but not limited to the following questions:

- Why did you choose this system and these component boundaries? Where are identity, authority, ownership, and dependency trust established? How do the three selected design principles appear in the implementation?
- How is a user or function authority carried through your workflow? Which checks prevent a permitted service from performing an unauthorized action. Using sequence diagrams can help with visualizations.
- (Optional, if you attempted the supply chain attack) Where did the malicious code enter, what could it access, and how did you contain it?
- How did the initial plan influence development and gate decisions? Which protections came from your language/runtime, libraries, application logic, and deployment configuration? What would need to change for a production release?
- Document the planted/known vulnerabilities.

Support your claims with evidence by linking them to the relevant commits, code, tests, logs, and gate results.

Submission

Submit the source repository and any artifacts (e.g., saved test and analysis results), the report, and an AI-use declaration (see below). The repository must include:

- A README with the application purpose and exact commands to build, seed, run, and test it.
- The initial lifecycle plan and the full Git history (i.e., you should not delete the .git folder).
- The architecture description, threat model, and access-control matrix.
- The Jenkins pipeline definition (e.g., the Jenkinsfile).
- The helper package (external dependency) and, if you attempted the supply chain attack, its malicious version and the approval manifest.

No marks will be awarded for this assignment. However, questions about the work may be asked during the oral examination.

AI usage policy

Please refer to the university's guidelines on the usage of AI https://studerende.au.dk/en/gai